

TSYP CS Challenge
Technical Report
AI Career Assistance Platform

Contents

1	Introduction and Goals	3
2	Technologies Used	5
3	CV Reviewer & Rewriter	6
3.1	CV Extraction System	6
3.2	RAG Pipeline	7
3.2.1	Embedding Model	7
3.2.2	PDF Loading & Chunking	7
3.2.3	Vector Store – ChromaDB	7
3.2.4	Retriever	7
3.2.5	LLM Extraction with Fallback	7
3.3	CV Reviewer	8
3.4	CV Rewriter	8
3.4.1	User Inputs & Outputs	8
3.4.2	Rewriting Prompt	8
3.4.3	Example JSON Output	8
4	Job Matching System	10
4.1	Goal	10
4.2	Embedding Model	10
4.3	Vector Store	10
4.4	Search & Ranking	10
5	AI Interviewer	11
5.1	Main Objectives	11
5.2	Interview Flow	11
5.3	RAG Module	11
5.4	Response Analysis	11
5.5	Expected Results	11
6	Cloud	12
6.1	Core Components	12
6.1.1	Google Cloud Storage (GCS)	12
6.1.2	Cloud Run (Fully Managed)	12
6.1.3	Firestore (Native Mode)	12
6.1.4	Firebase Authentication	13
6.1.5	Identity and Access Management (IAM)	13
6.1.6	Google Cloud Security Services	13
6.1.7	Auth0 (External Identity Layer)	13
6.2	Data Flow Overview	13
6.3	Security Principles	14

7	Screenshots	15
	Conclusion	17

Chapter 1

Introduction and Goals

This report presents the technical design and implementation details of the **AI Career Assistance Platform**, a modular, microservices-based, and scalable system built to help users improve employability through AI-powered features such as CV optimization, job matching, and interview preparation.

System Overview

The platform follows a **microservices architecture**, where each core functionality is implemented as an independent service. This design ensures scalability, maintainability, and ease of deployment. The platform is composed of five main services:

- **AI Interviewer:** Simulates and evaluates interview sessions using LLM-based questioning and scoring.
- **Job Matcher:** Scrapes, filters, and recommends relevant job postings using semantic similarity.
- **CV Reviewer:** Analyses user CVs, computes scores (quality, relevance, completeness), and generates personalized feedback.
- **CV Rewriter:** Generates improved and standardized CV versions.
- **Storage & Deployment Services:** Manages persistent storage, database access, and deployment pipelines for each microservice.

Additional Features

- **Front-end Customization:** Users can interact with the platform through an intuitive interface, with the ability to choose color themes and personalize the look and feel. This enhances user experience and engagement.
- **Economic Model:** The platform is designed to support a sustainable model, allowing free basic usage with premium features for advanced analytics, personalized feedback, and priority job matching.

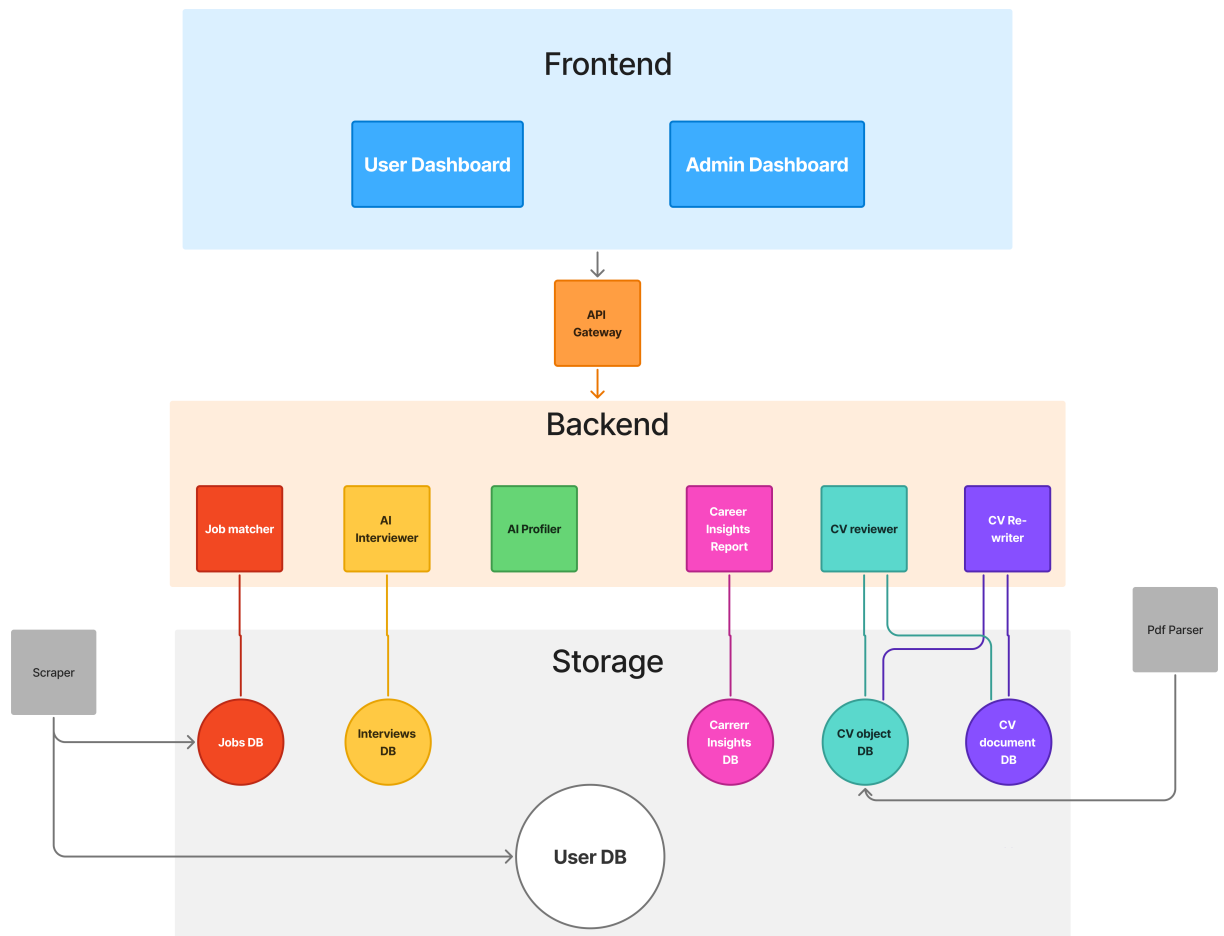


Figure 1.1: Global Architecture

Chapter 2

Technologies Used

Layer	Technologies
Backend	Flask, Python, LangChain
Frontend	React
Database	ChromaDB (CV & embeddings), Firestore
AI Layer	LLMs: nvidia/nemotron-nano-9b-v2, alibaba/tongyi-deepresearch-30b-a3b, Google Gemini-2 Embeddings: sentence-transformers/all-MiniLM-L6-v2, intfloat/e5-large-v2 Whisper & TTS: GTTS, pygame
Storage	Google Cloud Storage
Deployment	Google Cloud (GCP), Docker
NLP Libraries	LanguageTool, textstat

Chapter 3

CV Reviewer & Rewriter

3.1 CV Extraction System

Goal

Automatically extract structured information from user CVs, regardless of format or layout.

Extracted Fields

- Personal information (name, email, phone)
- Social links (LinkedIn, GitHub, portfolio)
- Professional summary
- Education (degree, school, dates, GPA)
- Experience (company, title, responsibilities, achievements)
- Projects, awards, certifications, volunteering, publications
- Skills (technical, soft, languages), hobbies & interests

Approaches Evaluated

Approach	Advantages	Disadvantages
Traditional NLP (Regex, NLTK, spaCy)	Fast, deterministic, low-cost, offline	Fragile to CV layouts, poor semantic understanding
RAG-based Extraction (LLM + Retrieval)	Excellent semantic extraction, works on any format	Slower, requires online LLM, higher cost

Choice: RAG-based extraction due to semantic accuracy and robustness.

3.2 RAG Pipeline

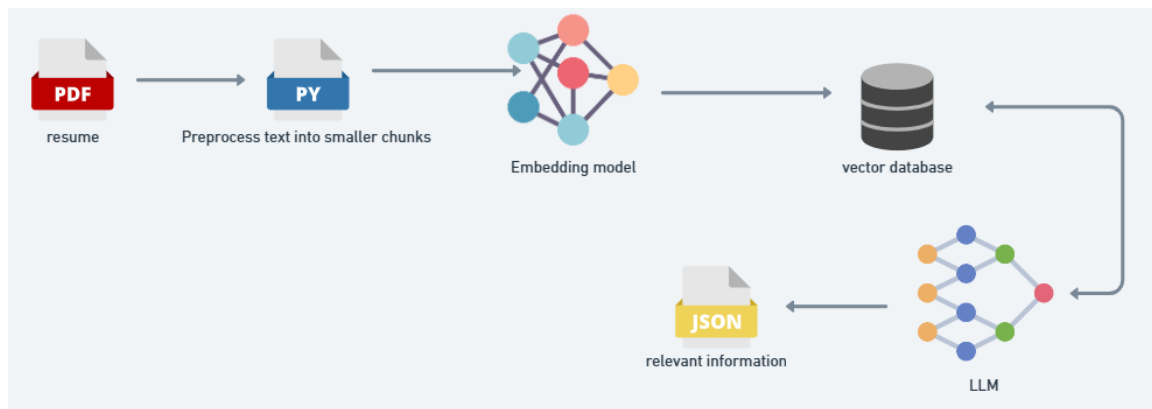


Figure 3.1: RAG-based CV Extraction Pipeline

3.2.1 Embedding Model

HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

3.2.2 PDF Loading & Chunking

```
1 loader = PyPDFLoader(DATA_PATH)
2 text_splitter = RecursiveCharacterTextSplitter(
3     chunk_size=1000, chunk_overlap=200
4 )
```

3.2.3 Vector Store – ChromaDB

All chunks are embedded and persisted in a Chroma collection.

3.2.4 Retriever

```
1 retriever = vector_store.as_retriever(search_kwargs={"k": 7})
```

3.2.5 LLM Extraction with Fallback

LLM used: alibaba/tongyi-deepresearch-30b-a3b

Example Prompt

```
1 You are an expert extraction algorithm.
2 Return structured JSON. If a field is unknown, return null.
3
4 Focus on:
5 {"personal_information": {
6     "full_name": "",
7     "email": "",
8     "phone": ""
9 }}
```

Fallback Steps

- clean_text()
- repair_json()

3.3 CV Reviewer

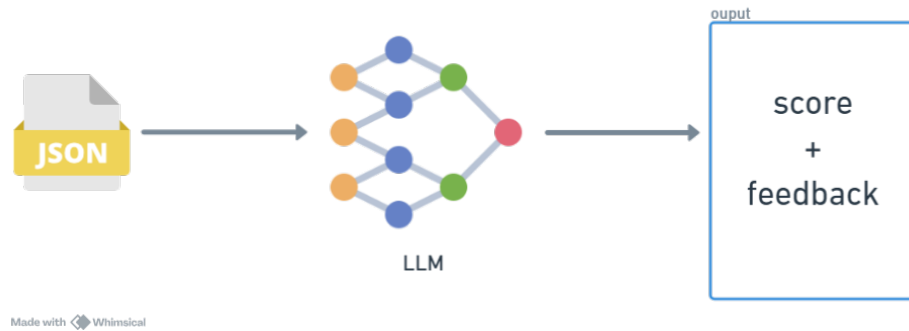


Figure 3.2: CV Reviewer Flow

Step 1 — Loading & Preprocessing

The CV is parsed from its JSON structure.

Step 2 — Relevance Score

Evaluates alignment with job title: skills fit, experience relevance, achievements.

Step 3 — Quality Score

Uses:

- LanguageTool grammar checks
- Readability (textstat)
- Structural penalties

Step 4 — Section Completeness

Mandatory: Experience, Education, Skills, Projects, Social links, Personal info.

Step 5 — Final Score

$$CVScore = 0.4Relevance + 0.4Section + 0.2Quality$$

3.4 CV Rewriter

3.4.1 User Inputs & Outputs

Inputs: PDF (text or scanned). **Outputs:** DOCX templates (Europass, Canadian, Minimalist).

3.4.2 Rewriting Prompt

```
1 SYSTEM: You are a CV-writing expert.
2 USER: Rewrite the following section in professional French:
3 {original_section}
```

3.4.3 Example JSON Output

```
1 {
2   "section": "experience",
```

```
3   "input": "...",  
4   "output": "...",  
5   "validation": {"coherence": true, "grammaire": 0.98}  
6 }
```

Chapter 4

Job Matching System

4.1 Goal

Match user CVs with the most relevant job descriptions using semantic retrieval.

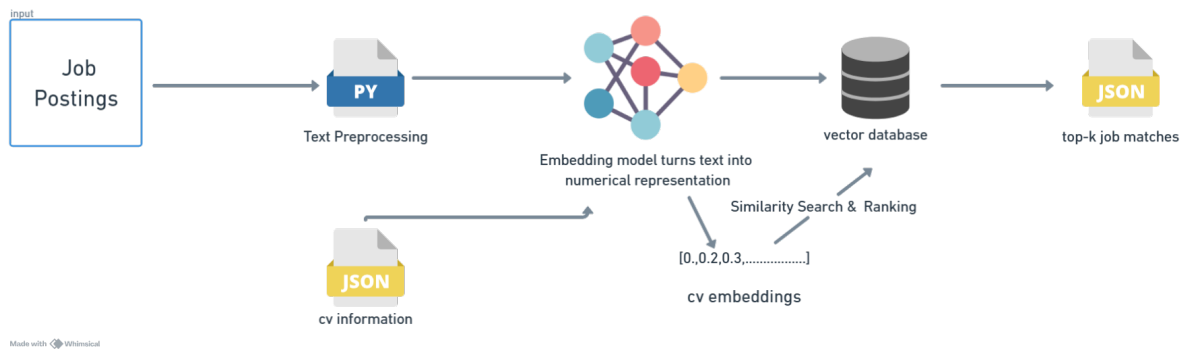


Figure 4.1: Job Matching RAG Pipeline

4.2 Embedding Model

we used the embedding model `intfloat/e5-large-v2` because The model performs well on both short and long text inputs and it is pretrained for semantic similarity and information retrieval tasks. Additionally, it offers a good balance between performance and computational efficiency,

4.3 Vector Store

Stored with metadata:

- `job_id`
- `title`
- `company`
- `location`

4.4 Search & Ranking

1. Compute CV embedding
2. Perform similarity search
3. Retrieve top 10 jobs
4. Rank by semantic similarity

Chapter 5

AI Interviewer

5.1 Main Objectives

- Simulate recruiter interviews
- Analyze quality of responses
- RAG-based context from CV
- Maintain dialogue memory

5.2 Interview Flow

Initialization

Load Gemini LLM, vectorize CV into ChromaDB.

Interview Loop

1. Record audio (Whisper)
2. Analyze response: empty, vague, confused, coherent
3. Retrieve CV context
4. Generate next question
5. Read aloud using GTTS

5.3 RAG Module

- Semantic retrieval of CV chunks (k=2-7)
- Ensures question coherence

5.4 Response Analysis

- Empty → reformulate
- Vague → simplify question
- Confused → clarify gently
- Coherent → move to next topic

5.5 Expected Results

- Natural, interactive interview simulation
- CV-based intelligent questioning Automatic clarifications

Chapter 6

Cloud

This chapter outlines the high-level design of our cloud-based system, built for scalability, security, and modularity to support CV processing, user data management, job matching, and AI-driven interview preparation. The architecture leverages Google Cloud Platform (GCP) services with integrated enterprise identity management via Auth0, ensuring robustness, compliance, and seamless user experiences.

6.1 Core Components

6.1.1 Google Cloud Storage (GCS)

Purpose: Centralized storage for raw and processed CV files.

Usage:

Stores original uploaded CVs (PDF/DOCX formats).

Hosts rewritten or reformatted versions of CVs after processing. **Security:** Data is encrypted at rest and in transit. Access is strictly controlled via Identity and Access Management (IAM) roles; public access is explicitly disabled.

6.1.2 Cloud Run (Fully Managed)

Purpose: Serverless execution environment hosting microservices as containerized applications.

Deployed APIs:

CV Rewrite API: Reformats and enhances CV content using NLP models.

CV Extract API: Parses structured data (skills, experience, education) from CVs into JSON.

Interview Prep API: Generates personalized mock interviews based on user profile and target job context.

Job Match & CV Scoring API: Matches user profiles with relevant job opportunities and computes fit scores using ML models. **Features:** Auto-scaling, stateless containers, secured via Identity-Aware Proxy (IAP) and mutual TLS for service-to-service authentication.

6.1.3 Firestore (Native Mode)

Purpose: NoSQL database for flexible, real-time storage of structured user and job-related data.

Stored Data Includes:

Parsed CV data (JSON format).

User profiles and preferences.

Job match results and scoring outputs.

Interview history and feedback logs. **Security:** Access is enforced through Firestore

security rules, ensuring that only authenticated users and authorized services can read or write data.

6.1.4 Firebase Authentication

Purpose: Handles user sign-in and identity management for end-users.

Integration: Supports email/password, Google Sign-In, and social identity providers.

Extension: Integrated with **Auth0** to enable advanced identity federation, multi-factor authentication (MFA), and custom identity workflows, facilitating enterprise-grade single sign-on (SSO) and compliance with organizational policies.

6.1.5 Identity and Access Management (IAM)

Role-Based Access Control (RBAC): Granular permissions are assigned to service accounts and users based on the principle of least privilege.

Example: Cloud Run services operate under dedicated service accounts with minimal permissions to access only required GCS buckets and Firestore collections.

Outcome: Ensures separation of duties across development, operations, and data access roles, reducing attack surface and enabling auditability.

6.1.6 Google Cloud Security Services

Secret Manager: Sensitive credentials (e.g., API keys, database passwords) are stored and rotated automatically, never hardcoded in source code or environment variables.

Security Command Center: Continuously monitors for threats, misconfigurations, and compliance violations across the environment.

VPC Service Controls: Defines security perimeters around critical services (Cloud Run, Firestore, GCS) to prevent data exfiltration and lateral movement.

Audit Logging: All administrative and data access activities are logged via Cloud Audit Logs for traceability and forensic analysis.

6.1.7 Auth0 (External Identity Layer)

Purpose: Acts as an enterprise-grade identity broker between Firebase Authentication and external identity providers (e.g., corporate SAML/OIDC systems).

Capabilities:

Enforces complex authorization policies and user lifecycle management.

Enables seamless integration with existing corporate directories and compliance frameworks.

Provides centralized control over MFA, session timeouts, and user provisioning/deprovisioning.

6.2 Data Flow Overview

The system operates through the following sequential workflow:

1. A user uploads a CV → stored securely in Google Cloud Storage (GCS).
2. The **CV Extract API** (Cloud Run) retrieves the CV, parses key fields, and writes structured JSON to Firestore.
3. The **CV Rewrite API** (Cloud Run) enhances the CV's format and content, then stores the improved version back in GCS.

4. The **Job Match & CV Scoring API** (Cloud Run) reads the user profile from Firestore, matches against job listings, and saves scoring results to Firestore.
5. The **Interview Prep API** (Cloud Run) generates personalized interview questions based on profile and job context, storing interaction history in Firestore.
6. All user interactions are authenticated via Firebase Authentication, with identity validation extended through Auth0 for enterprise users.
7. Backend services authenticate with each other using IAM service accounts and secure service-to-service communication protocols.

6.3 Security Principles

Zero Trust Model: Every request : whether from user or service is authenticated and authorized before access is granted.

Defense in Depth: Multiple layers of protection are enforced: IAM policies, VPC Service Controls, Secret Manager, encryption, and audit logging.

Compliance-Ready: The architecture is designed to meet regulatory requirements through data minimization, access controls, and audit trails.

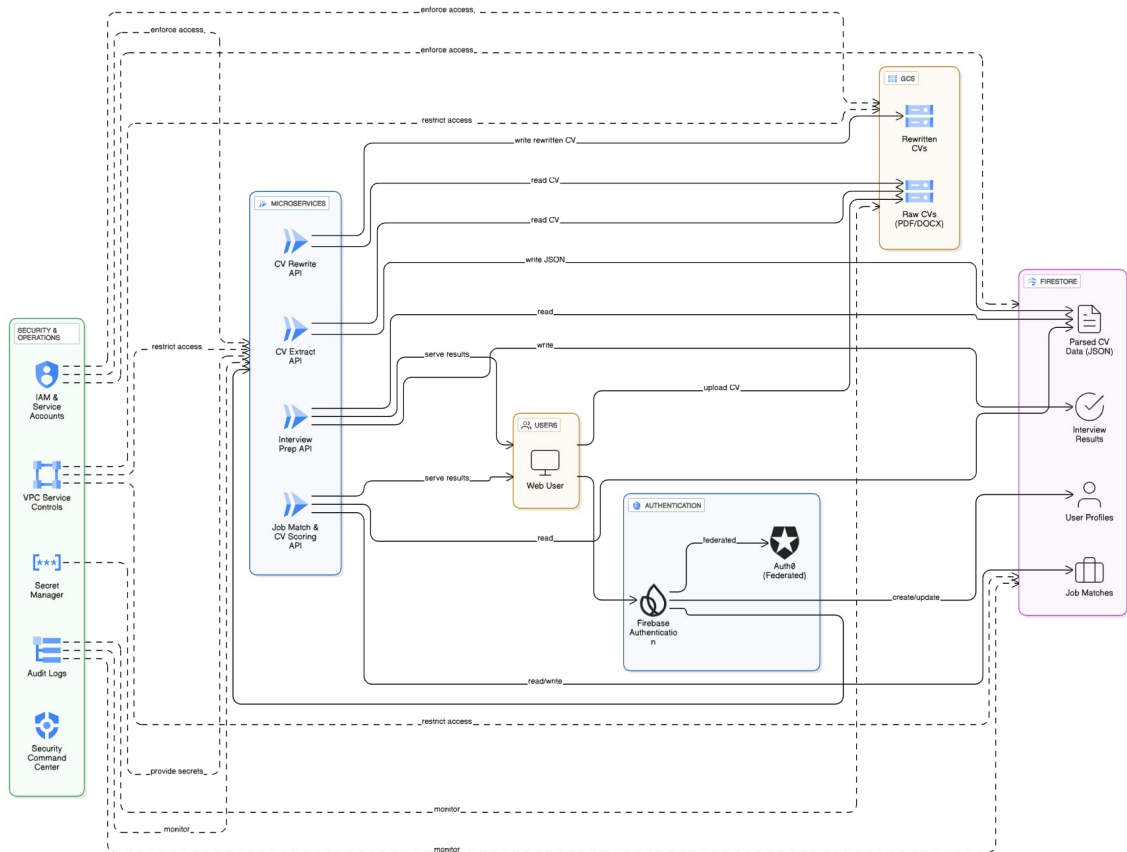


Figure 6.1: High-level cloud architecture diagram showing data flow and component interactions.

Chapter 7

Screenshots

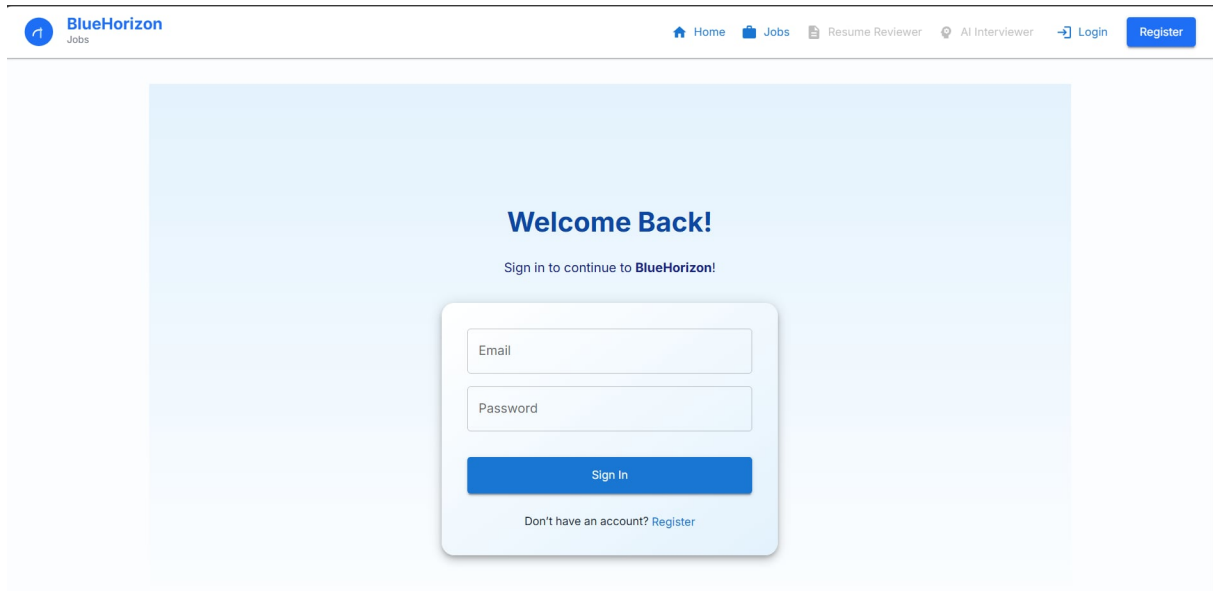


Figure 7.1: login interface

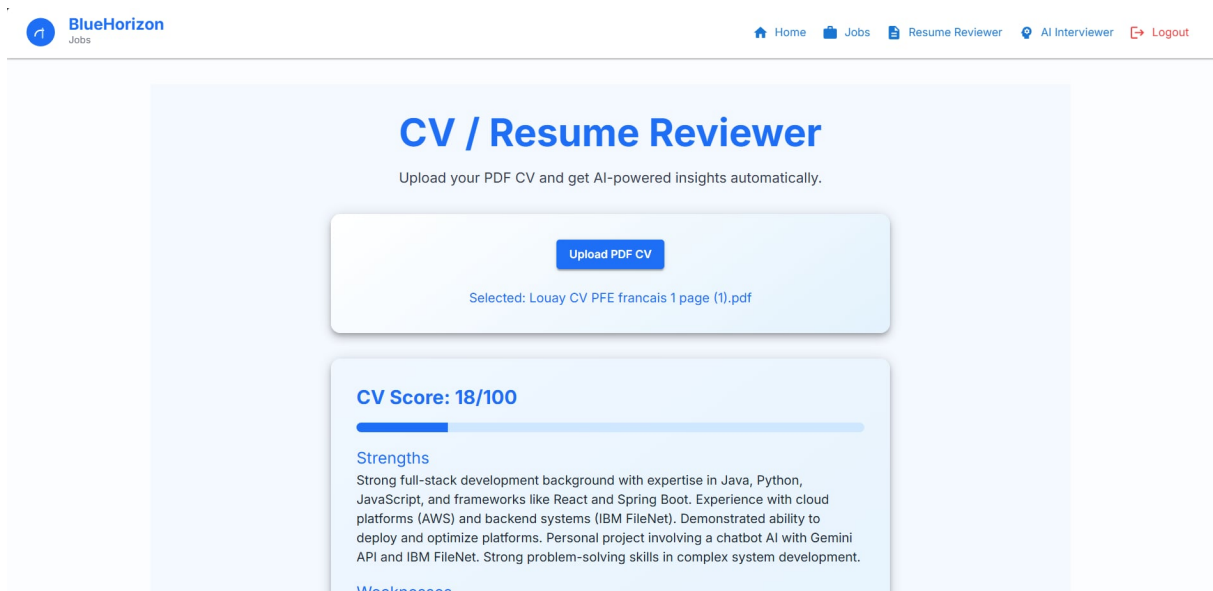


Figure 7.2: CV Reviewer Main Interface

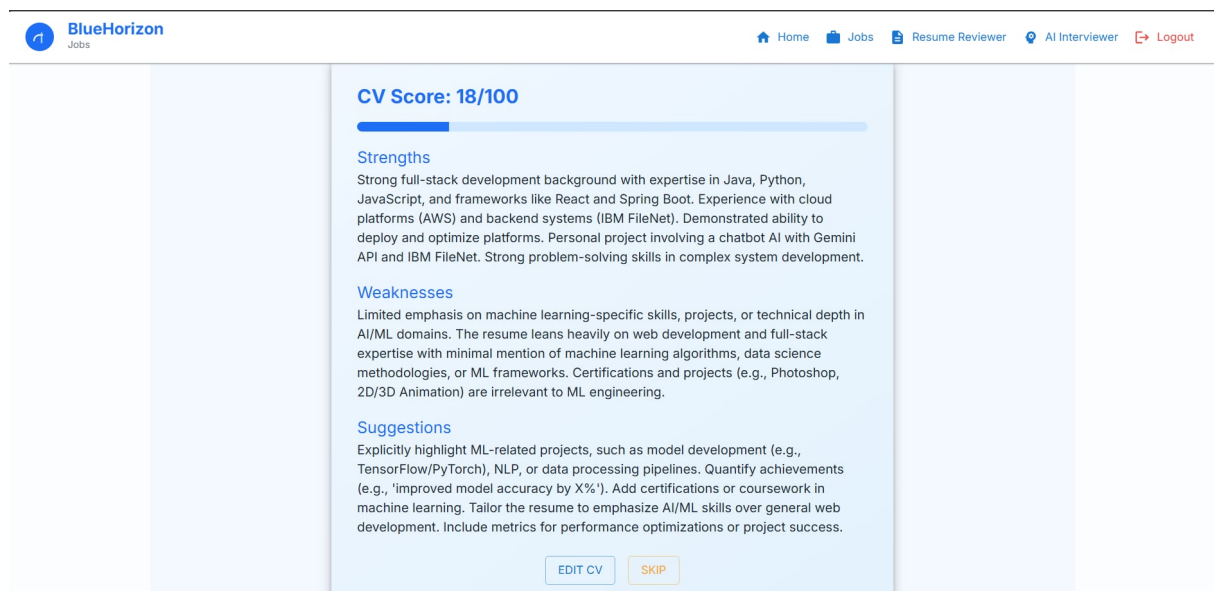


Figure 7.3: CV Analysis Results

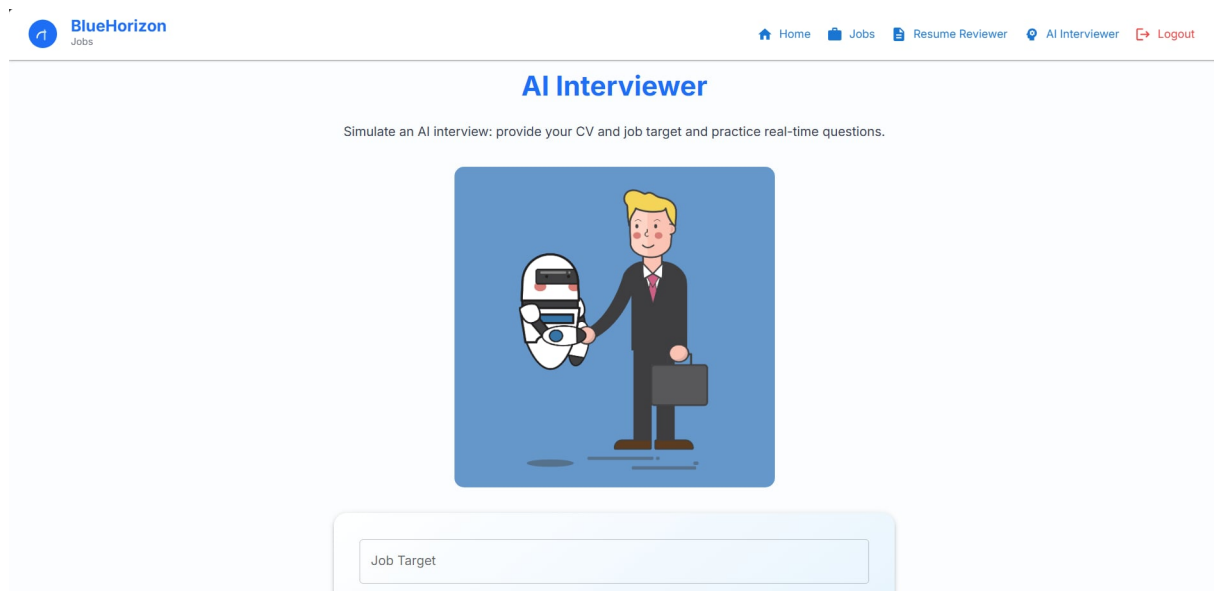


Figure 7.4: AI Interviewer interface

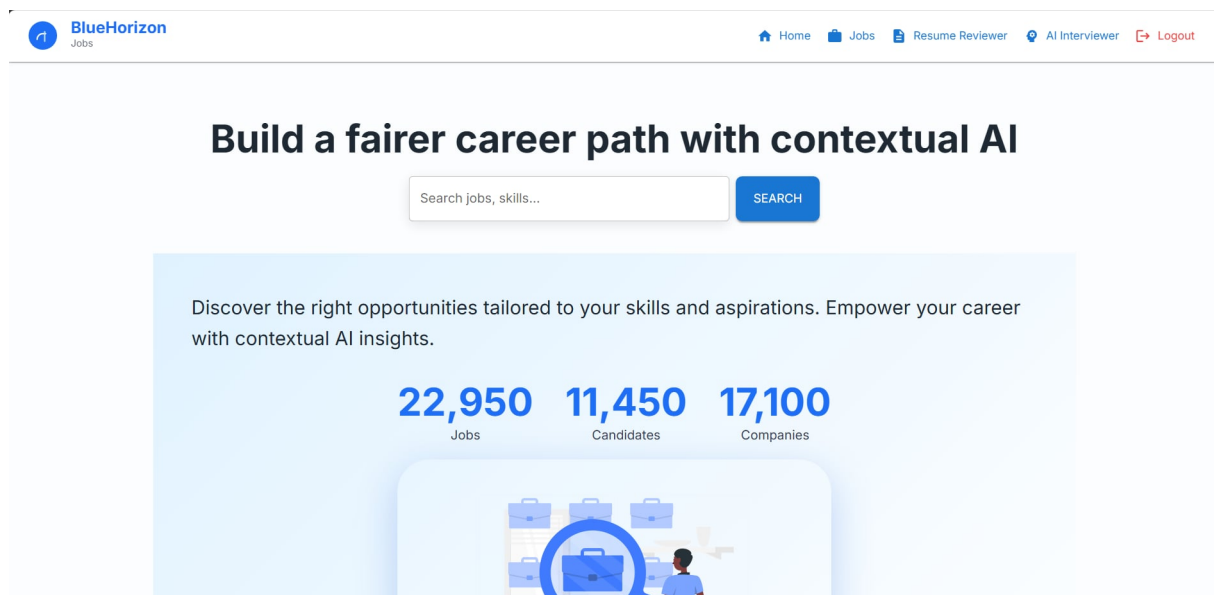


Figure 7.5: Job Matching interface

Conclusion

The **AI Career Assistance Platform** demonstrates a modular, AI-driven approach to enhancing employability. By combining RAG pipelines, semantic embeddings, and advanced LLMs, it can extract structured CV data, assess and improve CV quality, match candidates with relevant jobs, and simulate intelligent interviews.

The system balances accuracy, efficiency, and scalability, highlighting the power of modern NLP and retrieval-based AI techniques.